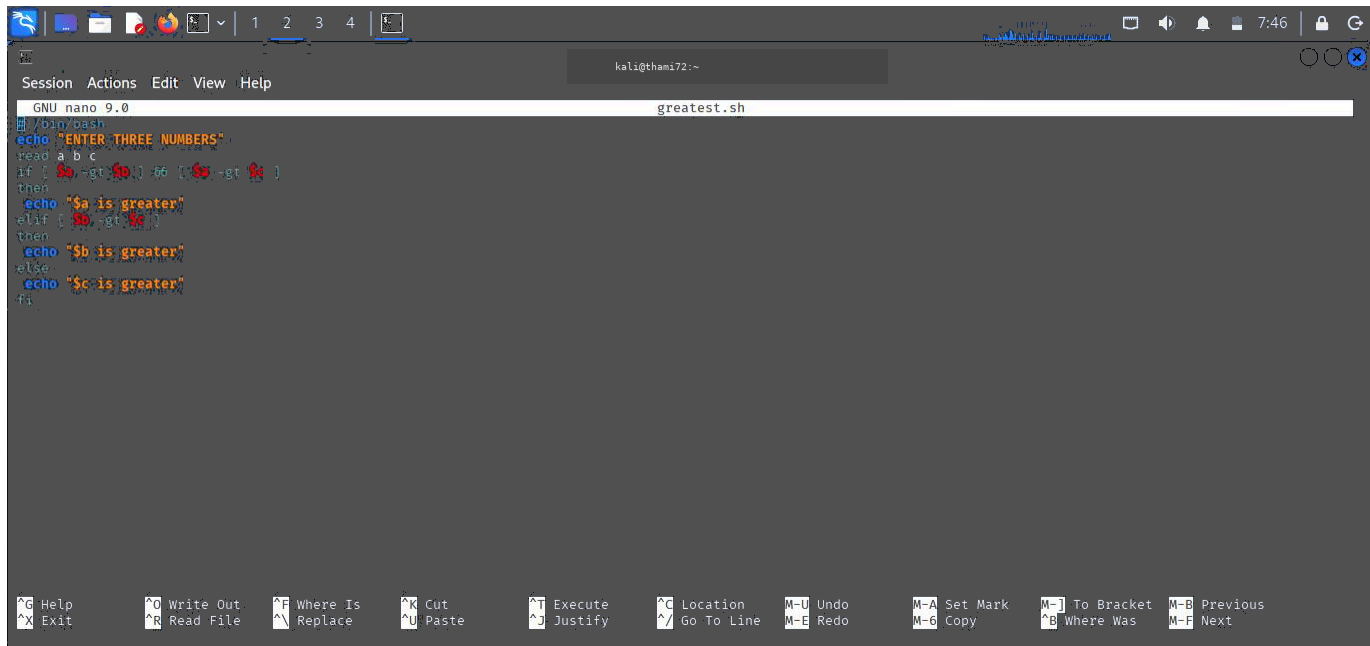


SHELL PROGRAMMING :GREATEST AMONG THREE NUMBERS

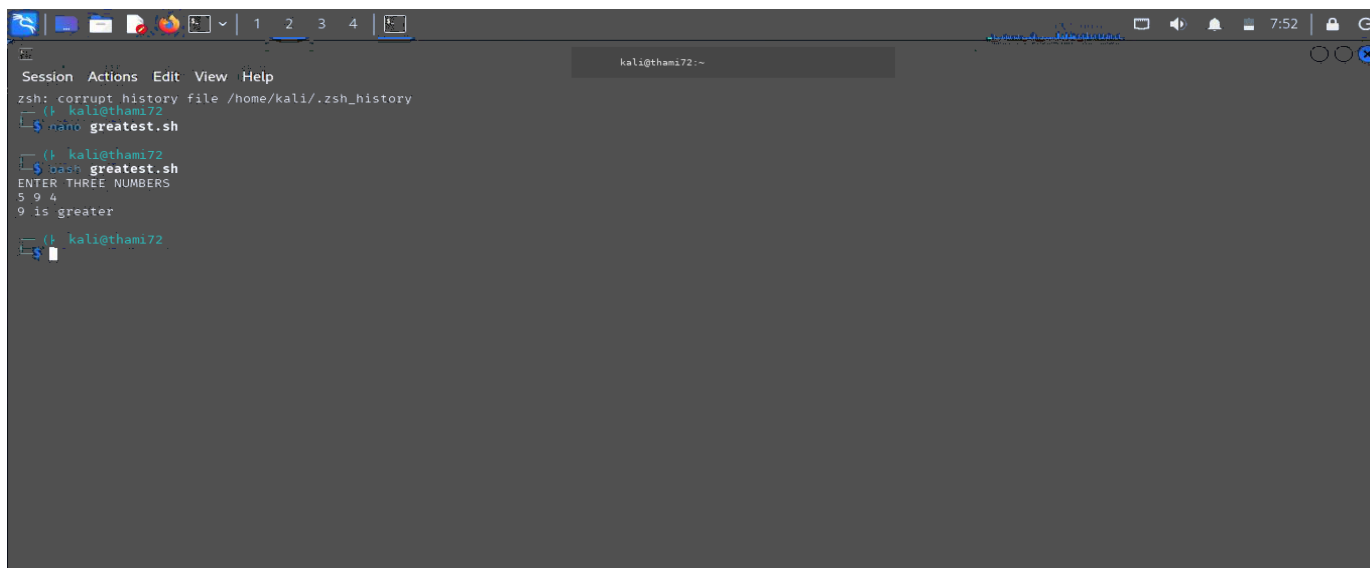


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'greatest.sh'. The script content is as follows:

```
#!/bin/bash
echo "ENTER THREE NUMBERS"
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]
then
echo "$a is greater"
elif [ $b -gt $c ]
then
echo "$b is greater"
else
echo "$c is greater"
fi
```

The terminal window title is 'kali@thami72:~'. The nano editor's status bar at the bottom shows 'GNU nano 9.0' and 'greatest.sh'. The bottom of the window displays various nano editor shortcuts like ^G Help, ^O Write Out, etc.

OUTPUT :

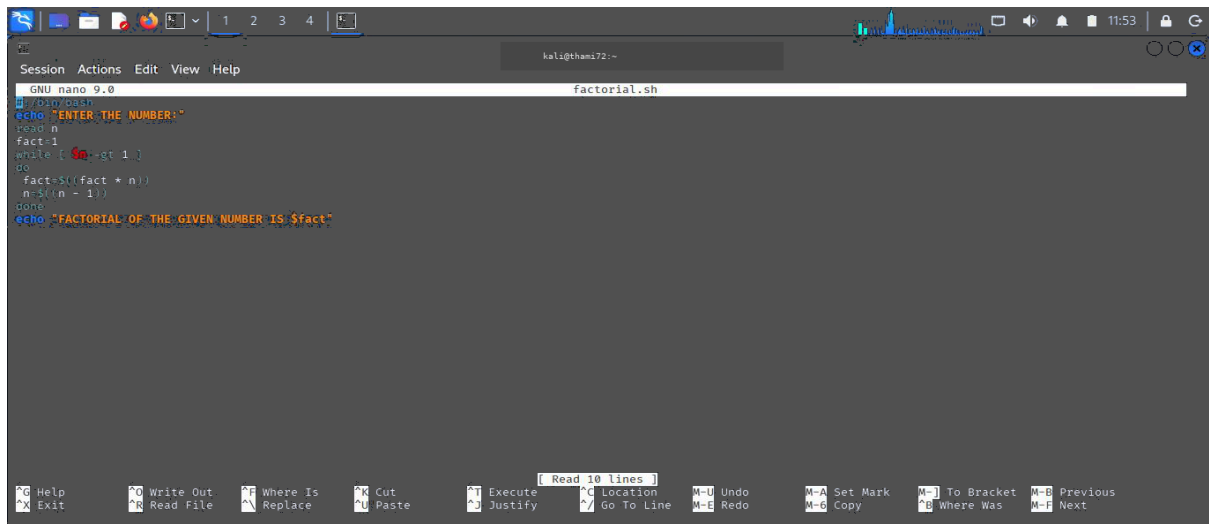


The screenshot shows the terminal window after running the script. The output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~
$ nano greatest.sh
kali@thami72 ~
$ bash greatest.sh
ENTER THREE NUMBERS
5 9 4
9 is greater
kali@thami72 ~
$
```

The terminal window title is 'kali@thami72:~'. The prompt is '\$'.

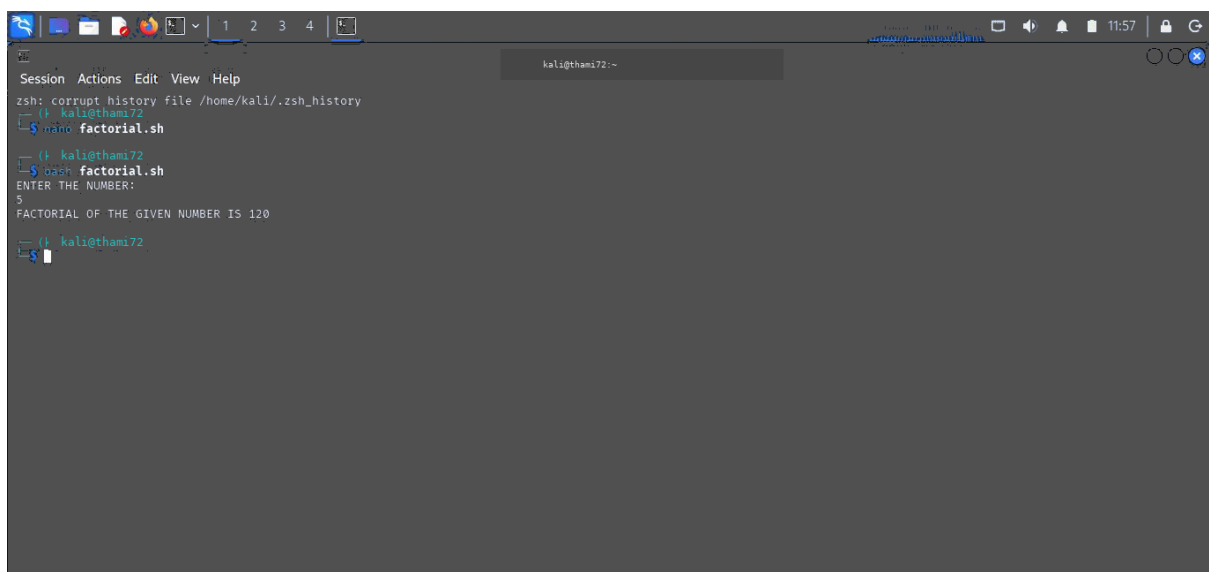
SHELL PROGRAMMING :FACTORIAL OF A GIVEN NUMBER



```
GNU nano 9.0 factorial.sh
#!/bin/bash
echo "ENTER THE NUMBER:"
read n
fact=1
while [ $n -gt 1 ]
do
    fact=$((fact * n))
    n=$((n - 1))
done
echo "FACTORIAL OF THE GIVEN NUMBER IS $fact"
```

The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'factorial.sh'. The script is a bash script that prompts the user to enter a number, reads it into a variable 'n', initializes a variable 'fact' to 1, and then uses a 'while' loop to calculate the factorial of 'n'. The loop continues as long as 'n' is greater than 1, multiplying 'fact' by 'n' and decrementing 'n' by 1 in each iteration. Finally, it prints the result of the factorial calculation.

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano factorial.sh
kali@thami72:~$ bash factorial.sh
ENTER THE NUMBER:
5
FACTORIAL OF THE GIVEN NUMBER IS 120
kali@thami72:~$
```

The screenshot shows a terminal window where the user has executed the 'factorial.sh' script. The prompt is 'kali@thami72:~\$'. The user enters '5' in response to the 'ENTER THE NUMBER:' prompt. The script outputs 'FACTORIAL OF THE GIVEN NUMBER IS 120'. The user's prompt returns to 'kali@thami72:~\$'.

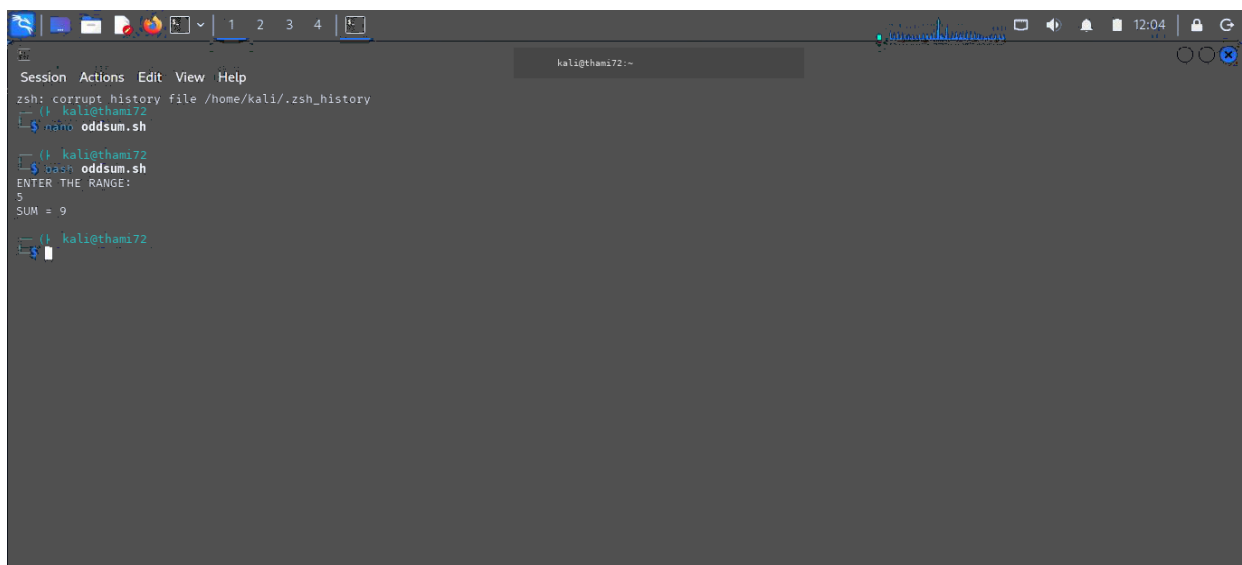
SHELL PROGRAMMING :SUM OF ODD NUMBERS



```
GNU nano 9.0 oddsun.sh
#!/bin/bash
echo "ENTER THE RANGE:"
read n
sum=0
x=1
while [ $x -le $n ]
do
    sum=$((sum + x))
    x=$((x + 2))
done
echo "SUM = $sum"
```

The screenshot shows a terminal window with the nano text editor open. The file being edited is named 'oddsun.sh'. The script contains a bash shebang, a prompt for the user to enter a range, a loop that iterates through odd numbers from 1 to the entered range, and a final echo statement showing the calculated sum. The nano editor's status bar at the bottom indicates 'Read 11 lines'.

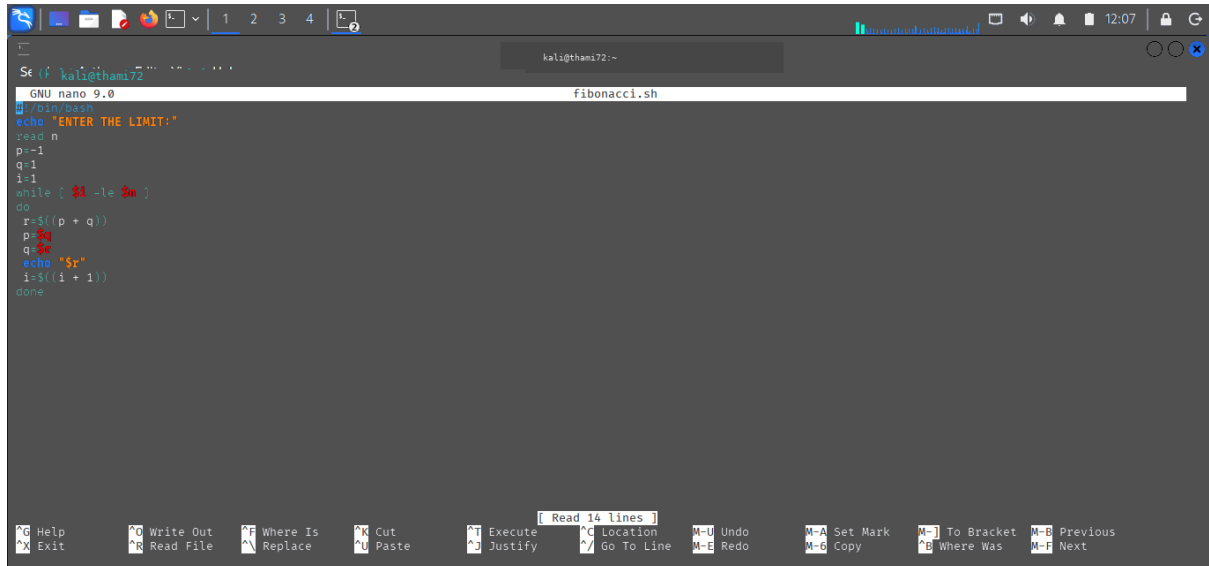
OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@tham172:~$ nano oddsun.sh
kali@tham172:~$ bash oddsun.sh
ENTER THE RANGE:
5
SUM = 9
kali@tham172:~$
```

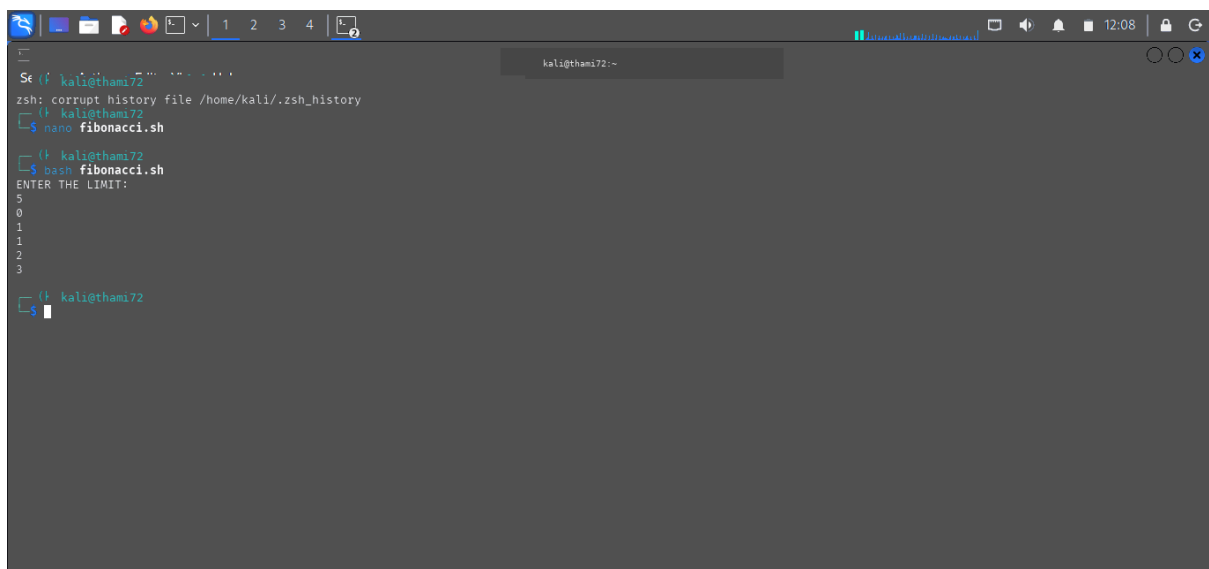
The screenshot shows the terminal output of the script. The user has run 'bash oddsun.sh', which prompted them to 'ENTER THE RANGE:'. They entered '5', and the script outputted 'SUM = 9'. The prompt returns to the shell.

SHELL PROGRAMMING :FIBONACCI NUMBER



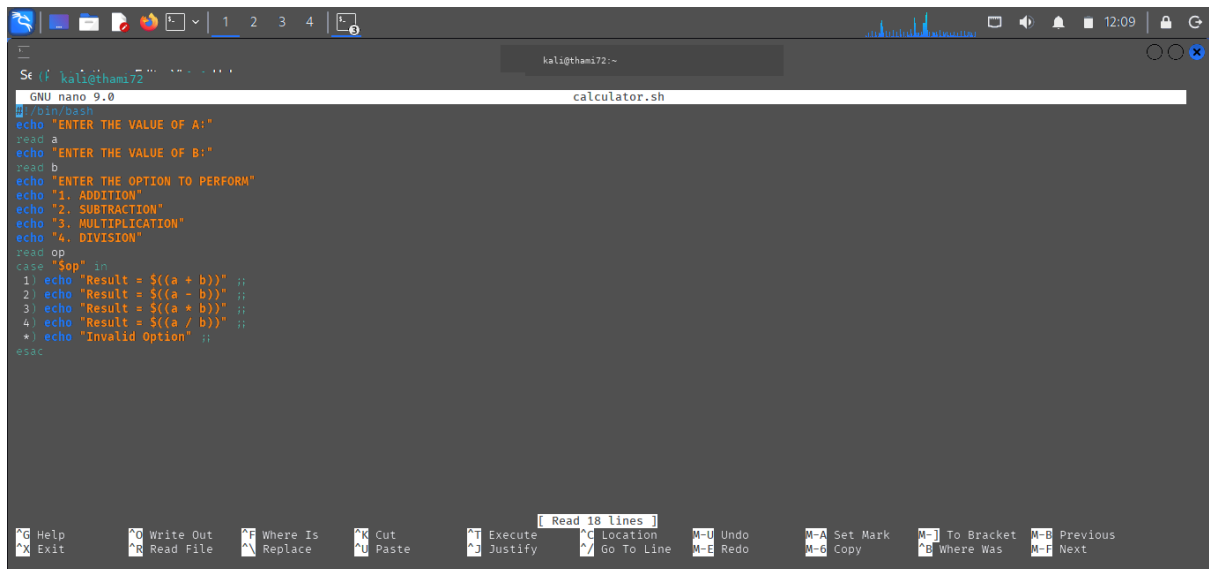
```
Se (f kali@thami72 ~)
GNU nano 9.0 fibonacci.sh
#!/bin/bash
echo "ENTER THE LIMIT:"
read n
p=1
q=1
i=1
while [ $i -le $n ]
do
r=$((p + q))
p=q
q=r
echo "$r"
i=$((i + 1))
done
```

OUTPUT :



```
Se (f kali@thami72 ~)
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~
$ nano fibonacci.sh
kali@thami72 ~
$ bash fibonacci.sh
ENTER THE LIMIT:
5
1
1
2
3
5
kali@thami72 ~
$
```

SHELL PROGRAMMING :ARITHMETIC CALCULATOR

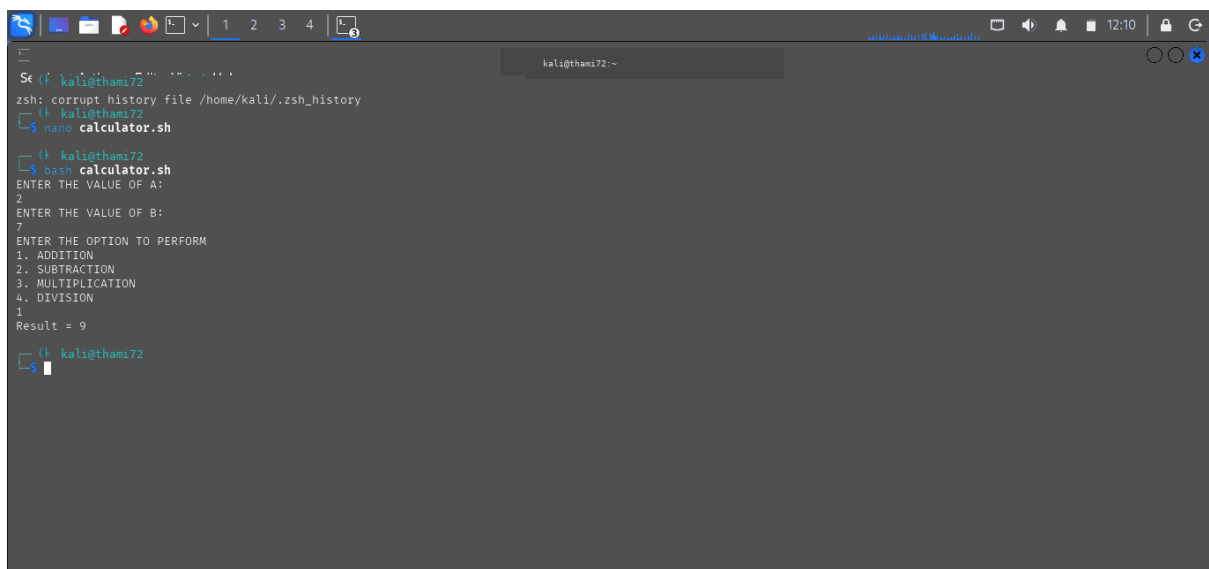


```
Se (f kali@thami72 ~)
GNU nano 9.0 calculator.sh
#!/bin/bash
echo "ENTER THE VALUE OF A:"
read a
echo "ENTER THE VALUE OF B:"
read b
echo "ENTER THE OPTION TO PERFORM"
echo "1. ADDITION"
echo "2. SUBTRACTION"
echo "3. MULTIPLICATION"
echo "4. DIVISION"
read op
case "$op" in
1) echo "Result = $((a + b))" ;;
2) echo "Result = $((a - b))" ;;
3) echo "Result = $((a * b))" ;;
4) echo "Result = $((a / b))" ;;
*) echo "Invalid Option" ;;
esac
```

Read 18 lines

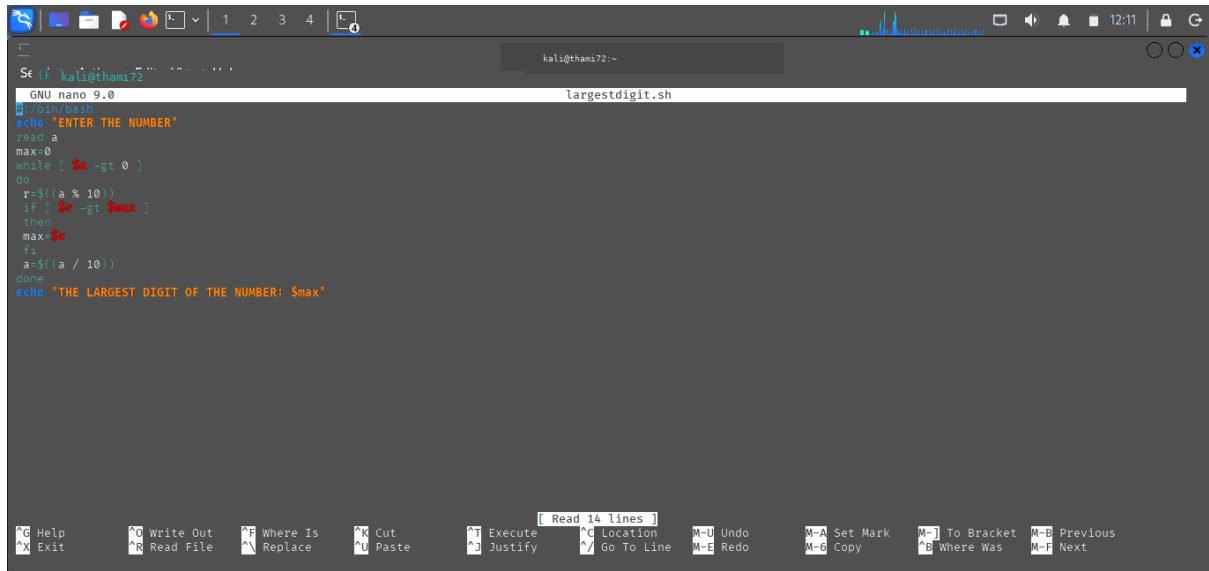
Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next

OUTPUT :



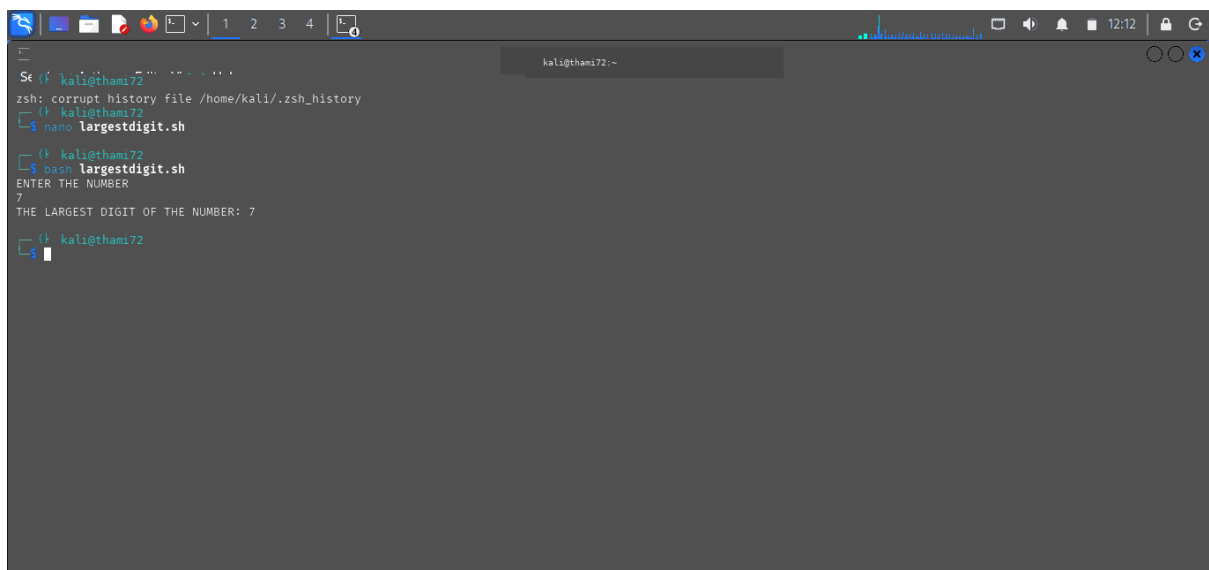
```
Se (f kali@thami72 ~)
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~
$ nano calculator.sh
kali@thami72 ~
$ bash calculator.sh
ENTER THE VALUE OF A:
2
ENTER THE VALUE OF B:
7
ENTER THE OPTION TO PERFORM
1. ADDITION
2. SUBTRACTION
3. MULTIPLICATION
4. DIVISION
1
Result = 9
kali@thami72 ~
$
```

SHELL PROGRAMMING :LARGEST DIGIT OF A NUMBER



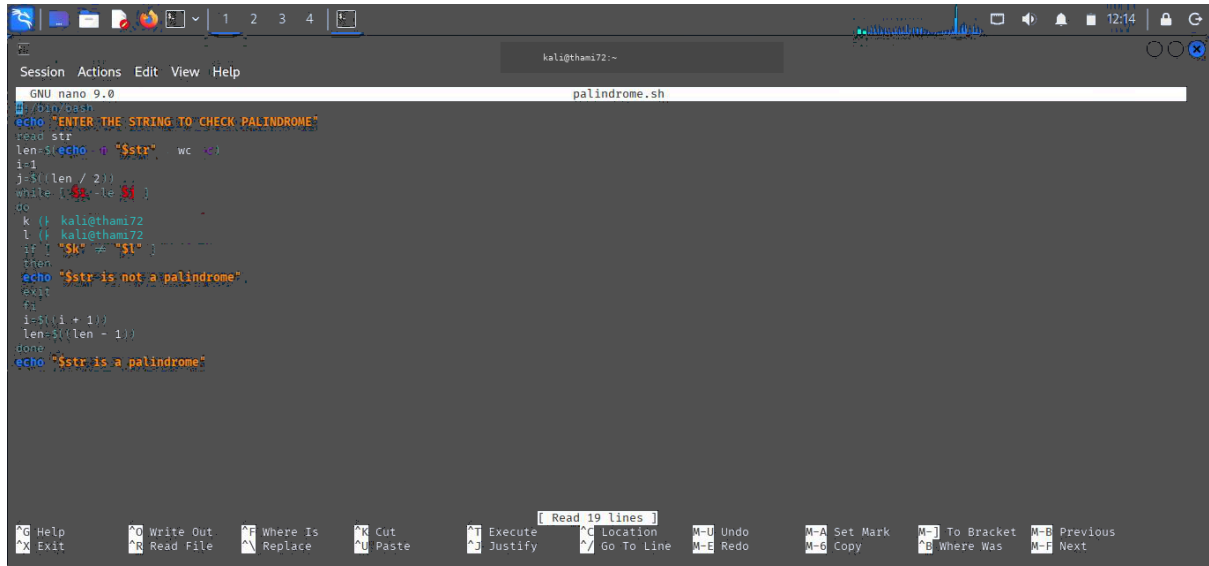
```
GNU nano 9.0 largestdigit.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read a
max=0
while [ $a -gt 0 ]
do
r=$((a % 10))
if [ $r -gt $max ]
then
max=$r
fi
a=$((a / 10))
done
echo "THE LARGEST DIGIT OF THE NUMBER: $max"
```

OUTPUT :



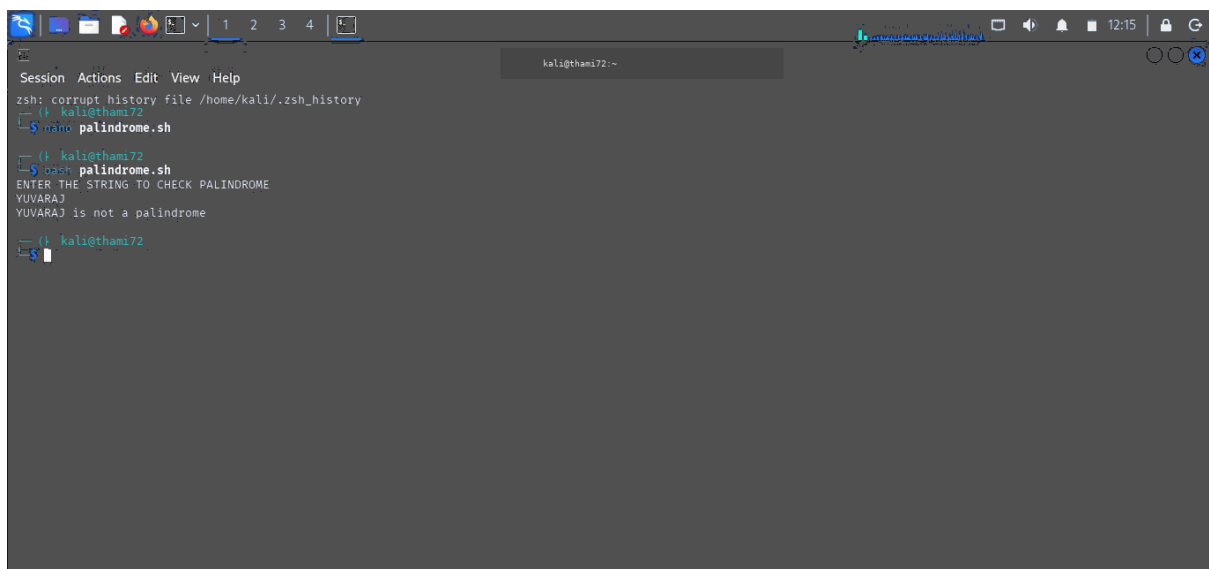
```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~
$ nano largestdigit.sh
kali@thami72 ~
$ bash largestdigit.sh
ENTER THE NUMBER
7
THE LARGEST DIGIT OF THE NUMBER: 7
kali@thami72 ~
$
```

SHELL PROGRAMMING :PALINDROME OR NOT



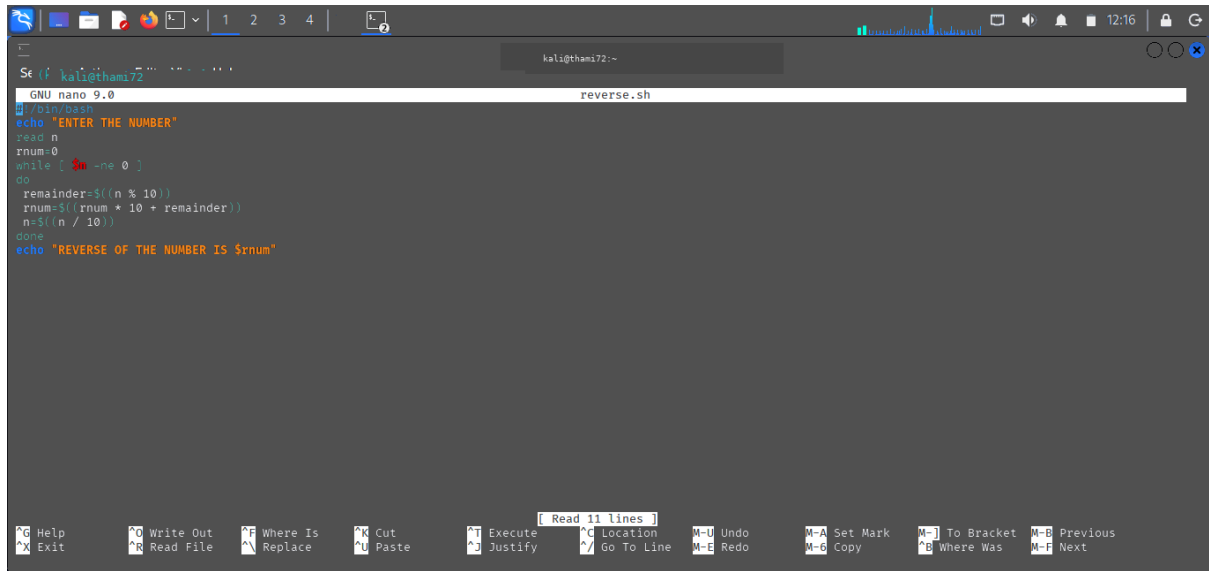
```
GNU nano 9.0
palindrome.sh
#!/bin/bash
echo "ENTER THE STRING TO CHECK PALINDROME"
read str
len=${#str}
i=1
j=$((len / 2))
while [ $i -le $j ]
do
    k=$((kali@thami72))
    l=$((kali@thami72))
    if [ "${str:$i:1}" != "${str:$l:1}" ]
    then
        echo "$str is not a palindrome"
        exit
    fi
    i=$((i + 1))
    j=$((j - 1))
done
echo "$str is a palindrome"
```

OUTPUT :



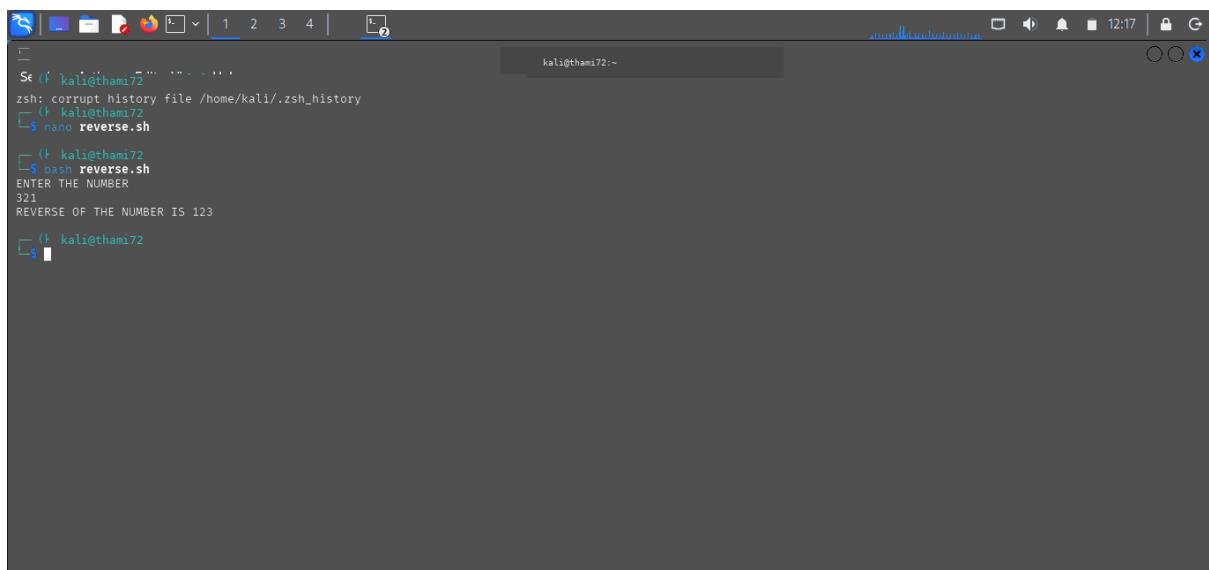
```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano palindrome.sh
kali@thami72:~$ bash palindrome.sh
ENTER THE STRING TO CHECK PALINDROME
YUVARAJ
YUVARAJ is not a palindrome
kali@thami72:~$
```

SHELL PROGRAMMING :REVERSE OF A GIVEN NUMBER



```
GNU nano 9.0 reverse.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read n
rnum=0
while [ $n -ne 0 ]
do
    remainder=$((n % 10))
    rnum=$((rnum * 10 + remainder))
    n=$((n / 10))
done
echo "REVERSE OF THE NUMBER IS $rnum"
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~
$ nano reverse.sh
kali@thami72 ~
$ bash reverse.sh
ENTER THE NUMBER
321
REVERSE OF THE NUMBER IS 123
kali@thami72 ~
$
```